Document number: P1320R1
Audience: EWG

Ville Voutilainen
2019-01-20

Allowing contract predicates on non-first declarations

Abstract

This paper proposes allowing contract predicates on non-first declarations for a member function. The rationale of doing so is, in a nutshell, to allow preconditions and postconditions to be implementation details. This thus also introduces the ability to redeclare a member function with a non-defining redeclaration.

Rationale

There are users that do not want to expose a contract checking mechanism in their class interfaces; there may be an English contract, but the language-level contract enforcement mechanism is an implementation detail that function declarations in a class definition should not expose.

It is probable that some would suggest that such use cases could be handled with contract assertions in function bodies. The reason why this paper proposes allowing contract predicates on non-first declarations is that

- In some cases, there is no body to put an assertion in; defaulted functions and pure virtual functions are examples of this.
- For postconditions, postconditions are guaranteed to be checked (when checking is enabled in the first place) at function exit; relying on contract assertions is more error-prone.

Usage examples

We basically propose being able to do this:

```
struct X
{
    void f();
};

void X::f() [[expects: foo]]
{
   ...
}
```

In addition, though, what is also proposed is this:

```
struct X
{
    void f();
```

```
};
void X::f() [[expects: foo]];

// definition in a separate translation unit
void X::f() [[expects: foo]]
{
   ...
}
```

Implementation impact, semantic restrictions

Allowing contract predicates on a non-first declaration probably means that in some such cases, the checking code can't be laid down at the call site unless the definition is also seen by the compiler.

This leads to an implementation concern about being able to decide whether all contract-checking is always done at the definition site, or whether it's possible to allow the caller to do that.

Based on an explanation by an implementation vendor, what we need to do is as follows:

1. If a class definition has a member function declaration *without* a contract on it, contract checking can be done at the call site if and only if the caller sees another declaration with a contract on it and the definition also sees such a declaration (possibly being one itself).
2. If a class definition has a member function declaration *with* a contract on it, the contract checking can be done at the call site regardless of other declarations.
3. If the member function declaration in the class definition did not have a contract, and an outside-of-class declaration visible to the caller has a contract, and there is no declaration visible at the site of the definition with a contract on it, the program is ill-formed, no diagnostic required.

Wording

In [class.mfct]/2, remove the redeclaration restriction:

> ~~Except for member function definitions that appear outside of a class definition, and except for explicit specializations of member functions of class templates and member function templates (12.8) appearing outside of the class definition, a member function shall not be redeclared.~~

In [dcl.attr.contract.cond]/1, modify as follows:

> A contract condition is a precondition or a postcondition.
> For a non-member function, ~~T~~the first declaration of a function shall specify all contract conditions (if any) of the function.
> For a member function, a member function redeclaration outside a class definition may specify contract conditions different from the ones in the declaration inside a class definition if the declaration inside a class definition had no contract conditions.
> Subsequent declarations shall either specify no contract conditions or the same list of contract conditions; no diagnostic is required if corresponding conditions will always evaluate to the same value.
> For a non-member function, ~~T~~the list of contract conditions of a function shall be the same if the declarations of that function appear in different translation units; no diagnostic required. For a member function, the list of contract conditions of a function shall be the same if

the redeclarations outside a class definition of that function appear
in different translation units; no diagnostic required.
[Note: a declaration in a class definition and a declaration
outside the class definition may have different lists of contract conditions. --end note]